

RFC 9767 (Rich Authorization Requests) vs GAuth (RFC 0111/0115)

RFC 9767 (Rich Authorization Requests) vs GAuth (RFC 0111/0115)

Executive Summary

RFC 9767 is an OAuth 2.0 extension that enables **fine-grained, structured authorization requests** using the `authorization_details` parameter.

GAuth is a Gimel Foundation framework focused on **legal delegation chains and Power of Attorney** for AI agents.

Key Finding: These frameworks address **different aspects** of OAuth 2.0 authorization but are **highly complementary**. GAuth provides the legal framework and authorization chains, while RFC 9767 provides fine-grained resource permissions.

1. What is RFC 9767 (RAR)?

RFC 9767: Rich Authorization Requests - Purpose: Extends OAuth 2.0 to support fine-grained, structured authorization requests - **Key Addition:** `authorization_details` parameter - JSON array describing specific authorization requirements - **Focus:** Resource-centric (what resources, actions, amounts, data types) - **Framework:** Works within existing OAuth 2.0 flow

Example RAR Request

```
{
  "authorization_details": [
    {
      "type": "payment_initiation",
      "actions": ["initiate", "status", "cancel"],
      "locations": ["https://example.com/payments"],
      "instructedAmount": {
        "currency": "EUR",
        "amount": "123.50"
      }
    }
  ]
}
```

2. High-Level Comparison

Aspect	GAuth (RFC 0111/0115)	RFC 9767 (RAR)
Primary Focus	Legal delegation chains & Power of Attorney	Fine-grained resource authorization
Authorization Model	Multi-party chains (3+ levels)	Single request with detailed permissions
Scope Expression	Structured authorization chains	<code>authorization_details</code> JSON array
Legal Framework	Commercial register, PoA credentials	Not addressed
Identity Verification	18-country national ID systems	Not addressed
Delegation Depth	3+ levels with validation	Single-level (user → app)
Token Structure	Extended tokens with PoA metadata	Standard OAuth tokens + fine-grained claims
Use Case	AI agents with legal authority	Banking, payments, healthcare APIs
Compliance	EU eIDAS, PoA laws, GDPR	Industry-specific (PSD2, FHIR)

3. Detailed Feature Comparison

3.1 Scope Expression

GAuth - Structured Authorization Chains

```
// Current GAuth approach
type AuthorizationScope struct {
    Read    []string // Resource types readable
    Write   []string // Resource types writable
    Admin   []string // Administrative actions
}

// GAuth Extended Token
{
  "power_of_attorney": {
    "issuer": "Owner's Authorizer",
    "grantee": "Client AI",
    "scope": {
      "transactions": ["payment", "contract_signing"],
      "geographic_scope": "EU",
      "value_limits": { "max": 10000 }
    }
  },
  "authorization_chain": [
    { "entity": "Owner's Authorizer", "authority": "Statutory" },
    { "entity": "Client Owner", "authority": "Delegated" },
    { "entity": "Client AI", "authority": "Granted" }
  ]
}
```

RFC 9767 - Authorization Details

```
{
  "authorization_details": [
```

```

{
  "type": "patient_record_access",
  "actions": ["read", "diagnose"],
  "locations": ["https://ehr.example.com/patients/789"],
  "data_types": ["medications", "lab_results", "imaging"],
  "purpose": "ai_diagnostic_assistance",
  "constraints": {
    "max_records": 100,
    "time_range": "2024-01-01/2025-12-31"
  }
}
]
}

```

3.2 Delegation Model

GAAuth - Multi-Level Authorization Chains

Owner's Authorizer (PAP)



Client Owner (PDP)



Client (AI)



Authorization Server validates entire chain



Extended Token with embedded chain



Resource Owner validates chain



Resource Server (demand-side PEP)

Characteristics: - 3+ levels of delegation - Each level verified (identity + authority) - Commercial register integration - Legal Power of Attorney credentials

RFC 9767 - Single-Level Fine-Grained Permissions

Resource Owner



Authorization Server



Token with authorization_details



Resource Server validates permissions

Characteristics: - Standard OAuth 2.0 flow - Detailed resource-level permissions - No delegation chain concept - Industry API compliance (PSD2, FHIR)

3.3 Token Structure

GAAuth Extended Token

```

{
  "access_token": "gauth_at_...",

```

```

"token_type": "GAuth-Extended",
"expires_in": 3600,
"power_of_attorney": {
  "poa_id": "poa_xyz789",
  "issuer": "Owner's Authorizer",
  "grantee": "Client AI",
  "scope": {...},
  "restrictions": { "value_limit": 10000 },
  "revocation_status": "active"
},
"authorization_chain": [
  { "entity": "Owner's Authorizer", "verified": true },
  { "entity": "Client Owner", "verified": true },
  { "entity": "Client AI", "verified": true }
],
"verification_proof": {
  "pvp_identity_check": true,
  "commercial_register_verified": true
},
"compliance_level": "rfc-0111-compliant"
}

```

RFC 9767 OAuth Token

```

{
  "access_token": "2YotnFZFEjr1zCsicMWpAA",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "read write",

  // RAR adds authorization_details to token introspection
  "authorization_details": [
    {
      "type": "payment_initiation",
      "actions": ["initiate"],
      "instructedAmount": {
        "currency": "EUR",
        "amount": "123.50"
      }
    }
  ]
}

```

4. Complementary Nature

GAuth + RFC 9767 Integration

GAuth could integrate RFC 9767 for richer authorization requests:

```

// Enhanced GAuth with RAR support
type ExtendedTokenRequest struct {
  // Existing GAuth fields
  GrantID      string
  PowerOfAttorney *poa.PoADefinition
}

```

```

    // RFC 9767 integration
    AuthorizationDetails []AuthorizationDetail `json:"authorization_details"`
}

type AuthorizationDetail struct {
    Type      string           // "payment_initiation", "patient_record"
    Actions   []string         // ["read", "update", "prescribe"]
    Locations []string         // Resource server endpoints
    Privileges []string         // Fine-grained permissions
    DataTypes []string         // Specific data types
    Metadata  map[string]interface{} // Context-specific
}

```

5. Use Case Matrix

Use GAuth (RFC 0111/0115) When:

AI agents need **legal authority** to act
Multi-party delegation chains required (Board → Company → AI → User)
Power of Attorney validation is critical
Commercial register verification needed
 EU regulatory compliance (**eIDAS, PoA laws**)

Examples: - Healthcare AI with guardian authorization - Corporate AI with board authority - Financial AI with statutory power

Use RFC 9767 (RAR) When:

Fine-grained resource permissions needed (payment amounts, transaction types)
 Standard OAuth 2.0 is sufficient
Industry APIs (PSD2 banking, FHIR healthcare)
 Complex scope requirements beyond simple strings
 No legal delegation required

Examples: - Banking APIs with transaction limits - Healthcare APIs with specific record types - Payment APIs with amount constraints

Use Both Together When:

AI agents with legal authority need **fine-grained permissions**
 Healthcare AI with PoA accessing **specific patient record types**
 Financial AI with statutory authority executing **specific transaction types**
 Corporate AI with board authorization performing **specific actions**

6. Practical Example: Combined Use Case

Scenario: Healthcare AI with Legal Power of Attorney

An AI diagnostic assistant needs to access specific patient records with legal authorization from a guardian.

Combined Request:

```

{
  "grant_id": "sub_abc123",

```

```

// GAuth: Legal authority chain
"power_of_attorney": {
  "poa_id": "poa_xyz789",
  "authorization_chain": {
    "levels": 3,
    "authorizer_id": "guardian_123",
    "client_owner_id": "hospital_456",
    "resource_owner_id": "patient_789"
  }
},

// RFC 9767: Fine-grained resource permissions
"authorization_details": [
  {
    "type": "patient_record_access",
    "actions": ["read", "diagnose"],
    "locations": ["https://ehr.example.com/patients/789"],
    "data_types": ["medications", "lab_results", "imaging"],
    "purpose": "ai_diagnostic_assistance",
    "constraints": {
      "max_records": 100,
      "time_range": "2024-01-01/2025-12-31"
    }
  }
]
}

```

Result: - **GAuth** validates the legal authority chain (Guardian → Patient → AI) - **RFC 9767** specifies exactly what data can be accessed and how - **Combined security:** Legal authority + fine-grained permissions

7. Implementation in GAuth

Current State

GAuth Today: - Structured authorization scopes (Read/Write/Admin) - Geographic scope restrictions
 - PoA credential embedding - **No `authorization_details` parameter**

Potential Enhancement

```

// pkg/gauth/authorization.go

type RichAuthorizationRequest struct {
  // Existing GAuth fields
  GrantID      string
  PowerOfAttorney *poa.PoADefinition

  // RFC 9767 integration
  AuthorizationDetails []AuthorizationDetail `json:"authorization_details"`
}

type AuthorizationDetail struct {
  Type      string `json:"type"`

```

```

    Actions      []string      `json:"actions,omitempty"`
    Locations    []string      `json:"locations,omitempty"`
    DataTypes    []string      `json:"datatypes,omitempty"`
    Identifier    string        `json:"identifier,omitempty"`
    Privileges    []string      `json:"privileges,omitempty"`
    Metadata     map[string]interface{} `json:"metadata,omitempty"`
}

```

This would make GAuth the first authorization server combining: - Legal delegation chains - Power of Attorney validation - Fine-grained resource permissions (RFC 9767)

8. Comparison Summary Table

Feature	GAuth	RFC 9767 (RAR)	Combined
Legal Authority	Full PoA support	Not defined	GAuth provides
Authorization Chains	Multi-level	Single-level	GAuth provides
Fine-Grained Permissions	Basic	Rich details	Best of both
Commercial Register	Integrated	Not defined	GAuth provides
Identity Verification	PVP (18 countries)	Not defined	GAuth provides
Resource Constraints	Value limits	Full constraints	Best of both
Data Type Filtering	Not supported	Supported	RAR provides
Action Granularity	Basic	Fine-grained	Best of both
OAuth 2.0 Compliance	Built on OAuth	OAuth extension	Full compliance
Industry Standards	Custom	IETF Standard	Standards-based

9. Integration Roadmap

Phase 1: Add authorization_details Support

```

// pkg/gauth/token_request.go

type TokenRequest struct {
    GrantID      string
    Scope        []string
    PowerOfAttorney *poa.PoADefinition
+   AuthorizationDetails []AuthorizationDetail `json:"authorization_details,omitempty"`
}

```

Phase 2: Validate Authorization Details Against PoA

```
func (v *ComplianceValidator) ValidateAuthorizationDetails(  
    poa *poa.PoADefinition,  
    details []AuthorizationDetail,  
) error {  
    // Ensure requested actions are within PoA scope  
    // Verify locations match authorized resources  
    // Check constraints against PoA restrictions  
}
```

Phase 3: Embed in Extended Token

```
{  
  "power_of_attorney": {...},  
  "authorization_chain": [...],  
  "authorization_details": [  
    {  
      "type": "payment_initiation",  
      "actions": ["initiate"],  
      "instructedAmount": {"currency": "EUR", "amount": "123.50"}  
    }  
  ]  
}
```

10. Conclusion

RFC 9767 and GAuth solve different problems but are highly complementary:

- **RFC 9767 (RAR):** Fine-grained resource permissions within OAuth 2.0
- **GAuth:** Legal delegation chains and Power of Attorney for AI agents

They are NOT competing - they work together: - GAuth provides the **legal framework** (authorization chains, PoA validation) - RFC 9767 provides **fine-grained resource permissions** (specific data types, actions, amounts)

Together, they offer unparalleled authorization control for AI systems with legal authority.

References

- RFC 9767 - Rich Authorization Requests
- OAuth 2.0 RFC 6749
- GiFo-RFC 0111 - GAuth 1.0 Authorization Framework
- GAuth Gap Matrix
- GAuth Architecture